

JavaScript is becoming the *de facto* platform for the Web

I remember a time in the late 90s when JavaScript developers wouldn't be taken seriously. If you developed in C or C++ (or better, if you programmed in Assembler), you were considered a serious developer. But if you were developing in PHP or JavaScript, you were ignored! Times have changed with the widespread adoption of the Web and the rising popularity of JavaScript's powerful frameworks and libraries. JavaScript is now used even in non-Web contexts—for example, in PDF documents. Its rising popularity, combined with its weaknesses, has prompted companies such as Google (with Dart) and Microsoft (with TypeScript) to attempt replacing JavaScript, but JavaScript continues to rule the roost.

Recently, I visited a company that develops software for set-top boxes and TVs, and was surprised at the extent that JavaScript code runs in these devices. There is a large list of languages that compile to JavaScript. There are also translators or adapted languages for major languages such as Java, C#, Ruby and Python that compile to JavaScript. I attended a talk on the Elm language and came to know that it compiles to JavaScript. I was amazed by what you can do with Elm, and I'll leave it to you to go and discover the magic of these new languages. I've also come across the Hack language, created by Facebook in an attempt to replace PHP. But it is too early to gauge whether this attempt has been successful or not. However, generally speaking, it is clear that JavaScript is here to stay.

Java and .NET: Emerging as target platforms for innumerable languages

Most language experts believe that compiled and statically typed languages have an edge over interpreted and dynamically typed languages when it comes to building large-scale enterprise applications. Statically typed languages such as Java, C# and C++ have long been used for creating commercial as well as open source applications. Though the Java platform was originally designed to run code written in the Java language, because of its all-pervasiveness, it has become a target platform for numerous other languages too. For instance, Scala compiles to the Java platform. Groovy extends Java and compiles to the Java platform. Languages such as Jython and JRuby are ports of Python and Ruby languages that target the Java platform. With its support for invokedynamic byte code and other features, it is easier for dynamic languages to compile to the Java platform.

Unlike the Java platform, the .NET platform was designed from the ground up to serve as a platform for a large number of languages. Other than C#, some important languages that translate to .NET are VisualBasic.NET, C++/CLI and F#. Older languages and other languages have ports to .NET as well — for example, Eiffel, IronPython and IronRuby. One of the disadvantages of .NET is that it primarily supports

the Windows platform. With Microsoft making the platform available for Linux and Mac OS, it can now be deployed in powerful Linux servers. And with Microsoft collaborating with Xamarin, .NET will be available for building mobile applications as well. With this development, the Java platform now has a serious competitor.

Dynamic languages are on the rise

Statically typed languages have ruled the language world for many decades (e.g., Fortran, C, C++, etc) but dynamically typed languages are slowly and steadily becoming more popular (Python, Ruby, Objective-C, etc). Dynamically typed languages have their own appeal and advantages. The quicker edit-run-debug cycle helps to develop code faster. Historically, dynamically typed languages are mostly scripting languages. However, there is no doubt that these languages are suitable for developing general-purpose applications. Dynamically typed languages also support meta-programming features (e.g., introspection) more easily than statically typed languages. Such benefits of dynamic typing have resulted in even statically typed languages acquiring some form of dynamism. For example, C# 4.0 introduced the 'dynamic' keyword. Languages like Groovy were inspired by the productivity offered by dynamic languages like Python and Ruby. Note that static typing has its own benefits - for instance, finding type conversions-related defects early in the development cycle. For this reason, even Groovy supports optional static type checking and static compilation. Given all this, the underlying general trend is clear - dynamically typed languages are on the rise.

Functional languages are becoming popular

From early languages like LISP to languages like Scheme and Haskell that appeared later, functional languages aren't new to the programming language community. However, what is surprising is the growing popularity of functional languages in the last 10 years. The main reason for this is concurrency. Writing correct concurrent code in the presence of mutable state is incredibly hard using libraries or language features in object-oriented as well as procedural languages. Functional programs don't change state. This characteristic makes the functional paradigm a suitable approach for writing concurrent programs. These days, even smartphones have multi-cores; so concurrency is here to stay.

New functional languages like Clojure have attracted wide attention in the programming community. Clojure is used extensively in Big Data circles, and its code can be compiled to run in JVM, .NET, or JavaScript. Other mainstream languages like C++, Java and C# have adopted functional programming features (mainly 'closures'/'lambda functions'). Many new languages (e.g., Elm) are functional programming languages. These developments indicate a clear trend - functional programming languages are becoming popular.